SIH26162

Team Leader Study Guide

AI-Based Detection and Classification of Industrial Fires and Persistent Thermal Sources Using NASA FIRMS, OSM & Satellite Data

Sponsor context: NTRO | Smart India Hackathon 2026 | Team ZeroOne

A personal technical handbook: go from "I barely understand this project" to "I can explain, defend, debug and present the whole system to judges."

> **HOW TO READ THIS**
> Every technical concept has four lines: **ELI5**, **OUR BUILD**, **WHY**, **MEMORY**.
> Status tags are used everywhere. Do not present a `[PLANNED]` feature as finished.

Tag	Meaning
`[IMPLEMENTED]`	Works today in the repo you can run right now.
`[PLANNED]`	In the approved plan / architecture.md, not yet coded.
`[OPTIONAL]`	Works only if an extra thing (e.g. an API key) is present; not required.
`[NOT SUPPORTED]`	Deliberately excluded. Must never be claimed.

Table of Contents

Part 1 — The Project in One Page

The project in one sentence

> **REMEMBER**
>
> We take NASA satellite heat detections over India, add industrial and geographic context, use a Random Forest to say whether each detection looks like a *persistent industrial source* or a *natural fire* (and flag the ones that fit neither as an *anomaly*), score how concerning it is, raise a prioritised, explained alert on a GIS dashboard, and let an operator ask questions about it in plain English.

Problem statement, in simple language

Satellites see **where** something hot is on the ground. They do **not** tell you what it is — a refinery flare, a steel plant, a forest fire, and stubble burning can all look similar in the raw feed. SIH26162 asks us to **separate industrial fires and persistent industrial thermal sources from natural / agricultural fires**, and to deliver it as a **GIS solution** (data stored and shown as map overlays).

Our solution (3–5 lines)

- Ingest NASA FIRMS thermal detections for India. `[IMPLEMENTED]`
- Enrich every detection with features: brightness temperature, FRP, persistence, day/night, distance to the nearest industrial facility, agri-season, land-cover zone. `[IMPLEMENTED]`
- Classify with a Random Forest (trained on global data, India held out) into `A` / `B_candidate`; flag low-confidence rows as anomalies. `[IMPLEMENTED]`
- Score risk (0–100) with a transparent rule engine, assign severity, raise alerts with a lifecycle, show them on a dark operations dashboard + India map + GIS export. `[IMPLEMENTED]`
- Reorganise the UI into clear sections and add a read-only natural-language **Fire Intelligence Agent**. `[PLANNED]`

THE MASTER MEMORY — the whole system as one chain

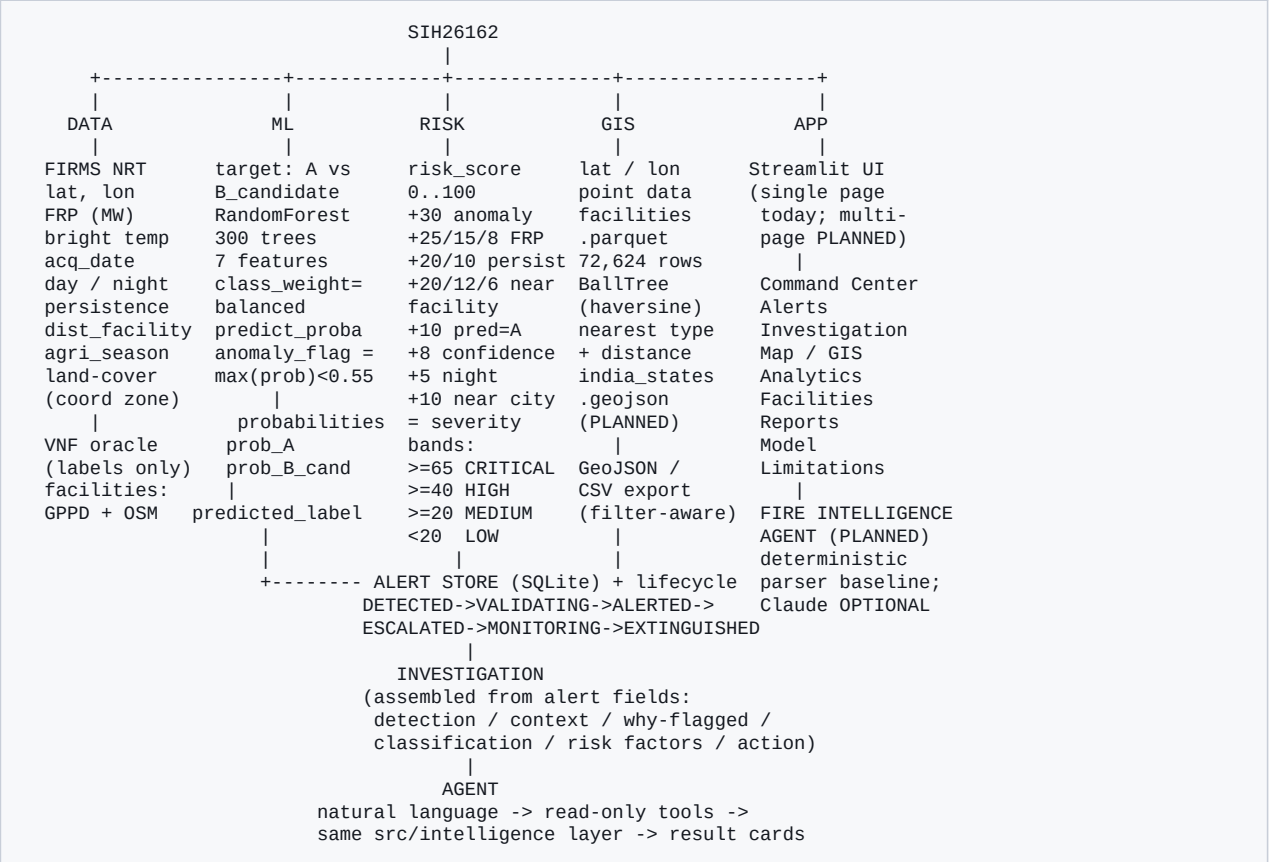
```
Satellite sensor detects heat on the ground
                    |
NASA FIRMS turns it into a DETECTION row (lat, lon, FRP, BT, time, day/night)
                    |
We ingest + clean it  (src/ingestion/)                      [IMPLEMENTED]
                    |
We add ENVIRONMENTAL + INDUSTRIAL context                   [IMPLEMENTED]
(persistence count, nearest facility + distance, agri-season, land-cover zone)
                    |
ML (Random Forest) estimates WHAT the thermal event is      [IMPLEMENTED]
-> predicted_label in {A, B_candidate} + probabilities + anomaly_flag
                    |
RISK ENGINE estimates HOW CONCERNING it is                  [IMPLEMENTED]
-> risk_score 0-100 -> severity CRITICAL/HIGH/MEDIUM/LOW -> narrative
                    |
GIS tells us WHERE it is and WHAT is nearby                 [IMPLEMENTED]
-> map marker, facility context, GeoJSON/CSV export
                    |
ALERT is generated + stored in SQLite with a lifecycle      [IMPLEMENTED]
                    |
OPERATOR investigates the alert (why was this flagged?)     [PLANNED page]
                    |
AGENT helps the operator ask questions in natural language  [PLANNED]
```

REMEMBER

Two verbs to never mix up: the **ML classifies** (what is it?), the **risk engine scores** (how much should I care?). They are different code, different logic.

Part 2 — Master Mind Map

Look at this for 30 seconds. It is the entire project.



REMEMBER

DATA feeds ML and GIS. ML + GIS feed the RISK engine and the ALERT. The ALERT feeds the DASHBOARD. The DASHBOARD is what the AGENT and the operator both drive.

Part 3 — Problem Statement

ELI5

Imagine a smoke alarm for the whole country, seen from space. It beeps whenever anything is hot. It cannot tell you if the hot thing is a factory doing its normal job, a factory on fire, or a farmer burning crop stubble. Our job is to be the person standing next to the alarm who says: "that one is a normal factory... that one is a crop fire... that one is weird, someone should go look."

Technical explanation

- **What SIH26162 asks:** (i) classify and segregate industrial fires from forest/natural fires; (ii) a GIS-based solution for storage + visualisation as map overlays. (from the dashboard docstring & `context.md`)
- **Why thermal anomalies matter:** industrial fires near refineries, chemical plants, power stations and mines threaten life, infrastructure and the environment; persistent-source monitoring supports compliance and activity intelligence.
- **Why facilities matter:** a hotspot's *meaning* depends on context — the same brightness reading 200 m from a refinery vs. in a forest are very different situations. Distance-to-facility is our strongest single feature (importance 0.29).
- **Why satellites alone are not enough:** NASA states MODIS/VIIRS active-fire products do *not* attribute anomaly type (fire vs flare vs volcano). A raw hotspot is never a labelled example of anything.

What NASA FIRMS gives us / does not give us

FIRMS PROVIDES	FIRMS DOES NOT PROVIDE
lat, lon of a thermal anomaly	the cause (fire? flare? kiln? volcano?)
brightness temperature (K)	a confirmed-fire label
Fire Radiative Power, FRP (MW)	industrial vs natural distinction
acquisition date + time	history older than ~5 days (NRT only)
day / night flag, confidence	sub-pixel detail (375 m / 1 km pixels)

REMEMBER

FIRMS gives us DETECTIONS, not complete explanations. Our system is the layer of intelligence that turns a detection into a decision.

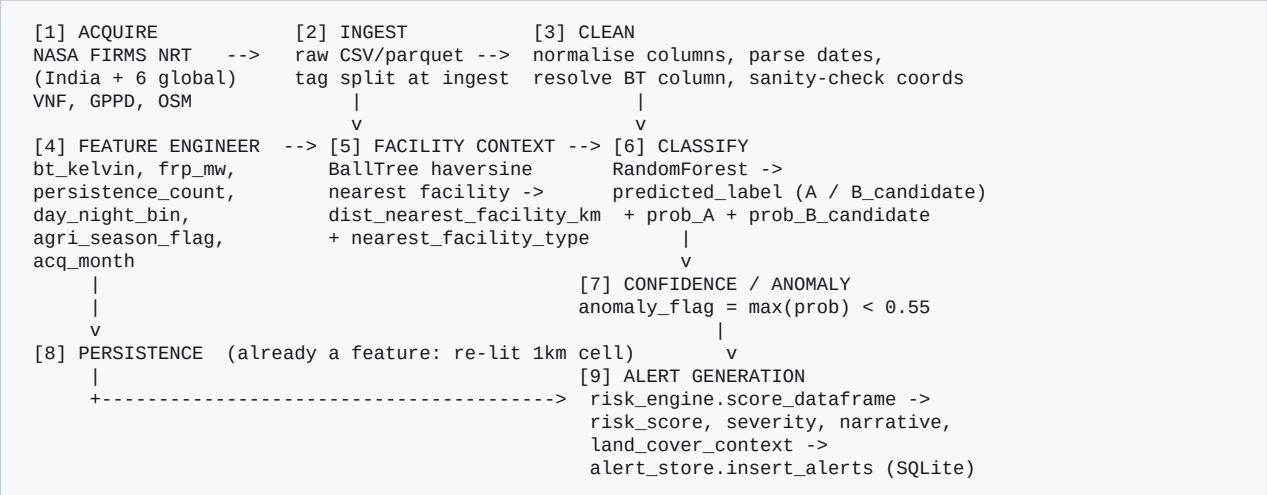
How to explain this to a judge in 30 seconds

EXAMPLE

"Satellites like NASA's FIRMS see heat anywhere on Earth, but they can't say what's burning. For an industrial region that's a problem: a refinery flare and a refinery fire look similar from orbit. We add industrial and geographic context to every detection, use a Random Forest trained on global data to separate persistent industrial sources from natural fires, flag the ones that match neither, score the risk, and put it on a GIS dashboard an operator can actually act on."

Part 4 — Data Pipeline

Our actual pipeline, stage by stage. Files are real paths in the repo.



Stage-by-stage

Stage	What / Why	In -> Out	File	Status
1 Acquire	Download FIRMS NRT (India + 6 non-India regions), VNF flare catalogue, GPPD + OSM facilities. Needed as raw material.	HTTP API -> raw CSV/parquet in data/raw/	src/ingestion/firms.py, vnf.py, facilities.py	[IMPLEMENTED]
2 Ingest	Write raw files, tag each row split (train_global / validation_global / india_holdout) at ingest time so it can't be faked later.	raw response -> tagged parquet	src/ingestion/*, src/model/split.py	[IMPLEMENTED]
3 Clean	Normalise MODIS/VIIRS columns, pick the brightness-temp column, parse acq_date, coordinate checks.	raw parquet -> tidy rows	src/features/engineer.py	[IMPLEMENTED]
4 Feature engineer	Compute the 7 model features + extras per detection.	tidy rows -> features_stage4.parquet (25 cols, ~419k rows)	src/features/engineer.py	[IMPLEMENTED]
5 Facility context	For each detection find the nearest facility (BallTree, haversine) -> distance + type. Context, never a label.	detection + facilities.parquet -> dist_nearest_facility_km, nearest_facility_type	src/features/engineer.py _query_nearest_facility	[IMPLEMENTED]
6 Classify	Random Forest predicts A vs B_candidate + probabilities.	7 features -> predicted_label, prob_A, prob_B_candidate	src/model/train.py (model), stage6_india_scores.parquet (output)	[IMPLEMENTED]
7 Confidence / anomaly	If the top probability < 0.55 the model is unsure -> anomaly_flag = 1.	probabilities -> anomaly_flag	src/model/train.py:59,101-109	[IMPLEMENTED]
8 Persistence	Count how many times the same ~1 km cell was re-detected in the window (the strongest physical discriminator per the literature).	detections -> persistence_count	src/features/engineer.py:99-109,326-343	[IMPLEMENTED]

Stage	What / Why	In -> Out	File	Status
9 Alert generation	Rule-based risk score + severity + narrative + land-cover; insert into SQLite with an initial lifecycle status.	scored rows -> data/alerts.db	src/alerting/risk_engine.py, alert_store.py, pipeline.py	[IMPLEMENTED]

The data fields we actually use — one line each

Field	One-line meaning
`latitude / longitude`	Where on Earth the hot pixel is.
`frp_mw`	Fire Radiative Power in megawatts — how much heat energy the fire is radiating.
`bt_kelvin`	Brightness temperature of the pixel in Kelvin (FIRMS 4 um channel, ~300-500 K).
`acq_date / acq_month`	When the satellite saw it; month drives the agri-season flag.
`day_night / day_night_bin`	Was it a day or night pass; encoded 1=day, 0=night.
`persistence_count`	How many times that same ~1 km cell re-lit in the detection window.
`land_cover_context`	Coarse land-use zone from a coordinate rule (industrial / cropland / forest / mining / coastal).
`nearest_facility_type`	Category of the closest industrial site (refinery, power plant, steel, mine...).
`dist_nearest_facility_km`	Straight-line distance to that closest facility.
`prob_A / prob_B_candidate`	Random Forest probability for each of the two classes (sum to 1).
`predicted_label`	The class the model picked: 'A' or 'B_candidate'.
`anomaly_flag`	1 if max(prob_A, prob_B_candidate) < 0.55 — model can't confidently pick either pattern.
`risk_score`	0-100 number from the rule engine; NOT from the ML model.
`severity`	CRITICAL / HIGH / MEDIUM / LOW band derived from risk_score.

Part 5 — NASA FIRMS (our data source)

ELI5	FIRMS is NASA's free feed of "something is hot here" pings from weather satellites.
OUR BUILD	We pull FIRMS NRT (near-real-time, last ~5 days) for the India bounding box (lat 6-37, lon 68-97.5) plus 6 non-India regions for training. src/ingestion/firms.py.
WHY	It's global, free, no access restriction, and it's exactly the data the problem statement names.
MEMORY	**FIRMS = free eyes in the sky. It sees heat, not meaning.**

Key FIRMS terms

Term	Meaning in our project
Thermal anomaly	A pixel measurably hotter than its surroundings — a 'detection'. Not necessarily a fire.
FRP (Fire Radiative Power)	Rate of radiated heat energy in MW. Higher FRP -> more intense thermal event. Feature frp_mw; risk +25/+15/+8 at >=30 / >=15 / >=5 MW.
Brightness temperature	Apparent temperature of the pixel from its infrared radiance (K). Feature bt_kelvin. FIRMS pixel BT (~300-500 K) is a different quantity from VNF flame temp (1500-2000 K).
Day / night flag	Which satellite overpass. Night detections are less polluted by solar noise; risk +5 for night.
Confidence	FIRMS' own low/nominal/high (or 0-100). risk +8 when high.
NRT vs archive	We use NRT only (~5 days). Historical archive is [NOT SUPPORTED yet] — this is why our timeline depth and incident date-matching are limited.

REMEMBER

FIRMS gives us DETECTIONS, not complete explanations. Everything 'intelligent' in the project is what WE add on top: context, classification, risk, GIS, alerts.

JUDGE QUESTION

Q: Why FIRMS and not your own satellite / another dataset?

A: It is the dataset the problem statement specifies, it is global, free, needs only a free API key, and it is the operational standard for active-fire monitoring. We hold India out of training and use the rest of the world's FIRMS data to learn the physical patterns.

Part 6 — Feature Engineering

The model sees ONLY these 7 numbers per detection (`src/model/train.py:49-57`). Nothing else.

Feature	Meaning	Why it helps	Example
`bt_kelvin`	4 um pixel brightness temperature (K)	Industrial heat vs vegetation fire sit in different (overlapping) temperature ranges	~330 K
`frp_mw`	Fire Radiative Power (MW)	Intensity; very high or very low FRP shifts the likely cause	9.5
`persistence_count`	Re-detections of the same ~1 km cell in the window	A flare/kiln re-lights every pass; a wildfire is bursty. Best physical discriminator	4
`dist_nearest_facility_km`	Distance to nearest known industrial site	Industrial context. Our #1 feature (importance 0.29)	0.8
`agri_season_flag`	1 if month in {10,11,4,5,7,8,9,1,2}	Crop-residue burning is seasonal; helps tag Class B	1
`day_night_bin`	1 = day, 0 = night	Night persistence points to a continuous industrial source. Importance 0.25	0
`acq_month`	Calendar month 1-12	Seasonality signal (currently 0 importance — NRT window not in burning season)	8

ELI5	We turn messy satellite rows into a tidy list of 7 numbers the model can compare.
OUR BUILD	src/features/engineer.py builds a 25-column table; src/model/assemble.py keeps the 7 model columns and encodes day/night.
WHY	Models need consistent numeric inputs; the same 7 features must exist at India inference time, so we only use FIRMS-native quantities.
MEMORY	**Raw data -> Features -> Model. The model never sees the raw row.**

WATCH OUT

Facility **distance** is a feature. Facility proximity is **never** used as a label. 'Near a factory' does not mean 'industrial fire'.

Part 7 — Machine Learning

What the model actually is

Question	Answer (from the code)
Target classification	2 classes: A = persistent industrial thermal source; B_candidate = natural / agricultural fire.
Labels come from	VNF spatial oracle: a global FIRMS row within 5 km of a known VIIRS Nightfire gas-flare site -> 'A'; every other global FIRMS row -> 'B_candidate'. NO confirmed-incident labels.
Features	bt_kelvin, frp_mw, persistence_count, dist_nearest_facility_km, agri_season_flag, day_night_bin, acq_month.
Model	sklearn Pipeline: SimpleImputer(median) -> RandomForestClassifier(n_estimators=300, min_samples_leaf=10, class_weight='balanced', random_state=42).
Training samples	270,238 rows (A=1,655 / B_candidate=268,583).
Validation samples	64,864 rows (A=246 / B_candidate=64,618) — spatial grid holdout.
India inference	705 held-out India rows scored -> stage6_india_scores.parquet.
Outputs per row	predicted_label, prob_A, prob_B_candidate, anomaly_flag.

Why Random Forest

- Tabular data with 7 features — trees are the natural fit, not deep nets.
- **Explainability** — we can show feature importance and per-alert reasoning to judges (the code comment literally says "explainability priority over accuracy").
- Robust to unscaled features, missing values (with the imputer), and heavy class imbalance (`class_weight='balanced'`).
- Tiny positive class (~1,900 'A' rows) — deep learning would overfit.

```
DATA (labelled global FIRMS rows)
     |
FEATURES  (7 numbers, median-imputed)
     |
RANDOM FOREST  (300 decision trees, each votes)
     |
PREDICTED LABEL      <-- majority vote:  'A'  or  'B_candidate'
     |
PROBABILITIES        <-- vote share:  prob_A + prob_B_candidate = 1
     |
CONFIDENCE / ANOMALY <-- if max(prob) < 0.55  ->  anomaly_flag = 1
```

> **EXAMPLE**
> Random Forest = asking 300 slightly-different decision-makers the same question and taking the combined vote. If the vote is close (no side >= 55%), we don't force an answer — we call it an anomaly for a human to review.

The 0.55 threshold (it IS in the code)

`ANOMALY_THRESHOLD = 0.55` in `src/model/train.py:59` and `src/scoring/score_incidents.py:46`. `anomaly_flag = int(max(prob_A, prob_B_candidate) < 0.55)`. It is a fixed rule, not a learned parameter, and it is applied *after* the forest votes.

Evaluation metrics we obtained (from reports/stage6_evaluation.txt)

Evaluation	Accuracy	Class A P / R / F1	B_candidate P / R / F1
Random split (inflated benchmark)	0.9725	0.14 / 0.80 / 0.24	1.00 / 0.97 / 0.99
Spatial holdout (HONEST — saved model)	0.9806	0.11 / 0.57 / 0.18	1.00 / 0.98 / 0.99
India geographic holdout (no labels)	n/a	predicted A=309, B_candidate=396	anomaly 59 / 705 = 8.4%

Leakage gap: overall accuracy barely moves, but **Class A F1 drops 0.24 -> 0.18** when the split respects spatial grouping — that drop is the honest signal that random splitting leaks. Feature importance: dist_nearest_facility_km 0.29, day_night_bin 0.25, bt_kelvin 0.21, persistence_count 0.14, frp_mw 0.10, agri_season_flag 0, acq_month 0.

What the model does NOT claim

WATCH OUT

It never says "confirmed industrial fire". There is no dataset of confirmed industrial fires anywhere. The model separates two *learned patterns*; a real industrial incident often matches neither, so it is surfaced as an **anomaly for human review**.

Judge questions you must answer

JUDGE QUESTION

Q: Why Random Forest, why not deep learning / a CNN?

A: 7 tabular features and a ~0.6% positive class. Trees fit tabular data, need no scaling, handle imbalance with class weights, and are explainable. A CNN needs image inputs and far more labelled data than exists. We chose explainability over a marginal accuracy gain.

JUDGE QUESTION

Q: What are your labels and are they ground truth?

A: They are proxy labels, and we say so. 'A' = FIRMS row within 5 km of a VIIRS Nightfire gas-flare site; 'B_candidate' = other global FIRMS. No confirmed-fire labels exist. That's the core scientific honesty of the project.

JUDGE QUESTION

Q: How do you evaluate it?

A: Three ways side by side: random split (0.9725, inflated), spatial-grid holdout (0.9806, honest), and India geographic holdout (no labels, distribution + 8.4% anomaly rate). We report the gap, not hide it.

JUDGE QUESTION

Q: How accurate is the model, really?

A: Overall accuracy ~98% is dominated by the easy majority class. The honest number is Class A F1 = 0.18 on spatial holdout — the minority class is hard because we only have ~1,900 positive examples. Adding a historical FIRMS archive or GIHS would improve it.

JUDGE QUESTION

Q: What happens when the model is uncertain?

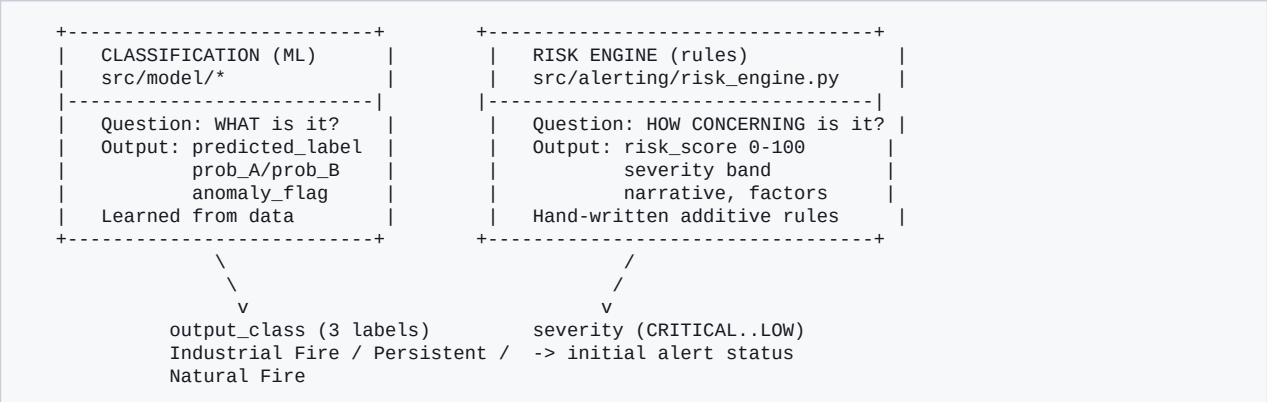
A: If neither class reaches 0.55 probability, anomaly_flag = 1 and the detection is presented as 'Industrial Fire / Abnormal Thermal Event' — explicitly a candidate for human verification, not a conclusion.

JUDGE QUESTION

Q: Can you guarantee an alert is a real fire?

A: No, and we never claim it. Every alert requires human verification. We detect anomalous departures from known patterns.

Part 8 — Classification vs Risk (critical distinction)



EXAMPLE

The ML might say "this looks like Class A (persistent source), prob 0.72". The risk engine independently says "FRP 40 MW + 4 repeat detections + 0.8 km from a refinery + near a city = risk 82 = CRITICAL". One tells you the *type*, the other tells you the *urgency*.

REMEMBER

Classification != Risk. A correctly-classified routine flare can be LOW risk. An anomaly near a chemical plant is CRITICAL risk. Judges love this distinction — it shows the system is operational, not just a classifier.

How the 3 dashboard classes are produced

```
# src/alerting/risk_engine.py  (score_row)
if anomaly_flag == 1:            output_class = "Industrial Fire / Abnormal Thermal Event"
elif predicted_label == "A":     output_class = "Persistent Industrial Thermal Source"
else:                            output_class = "Forest / Agricultural Fire"
```

So the model predicts 2 classes; the third ("Industrial Fire") is the anomaly bucket, assigned by a rule, not by the model.

Part 9 — Risk Engine

Everything here is literally in `src/alerting/risk_engine.py` function `score_row`. Start at 0, add points:

Signal (actual condition)	Points added
`anomaly_flag == 1`	+30
`frp_mw >= 30 / >= 15 / >= 5`	+25 / +15 / +8
`persistence_count >= 4 / >= 2`	+20 / +10
`dist_nearest_facility_km < 1 / < 5 / < 15`	+20 / +12 / +6
`predicted_label == 'A'`	+10
`FIRMS confidence is high ('h' / 'high' / >= 70)`	+8
`day_night == 'N' (night)`	+5
`nearest major city < 30 km`	+10

Severity bands (score -> band)

Band	Score	Meaning
CRITICAL	>= 65	Anomalous + persistent + near facility/population
HIGH	>= 40	Anomalous OR high FRP OR persistent near infrastructure
MEDIUM	>= 20	Moderate signal — monitor
LOW	< 20	Single low-confidence detection

Also produced by the risk engine (not ML)

- **land_cover_context** — coordinate-zone rule: 'Industrial Land Use' if <2 km & persistence>=2; else cropland / forest / mining corridor / coastal / mixed zones by lat-lon box.
- **hazard_facility_type** — maps the raw facility type string to one of: Oil Refinery / Petrochemical, Thermal Power Plant, Steel / Metal, Mining / Extraction, LNG / Gas Terminal, Chemical / Pharmaceutical, Brick Kiln / Ceramic, Nuclear Power Plant, or generic Industrial Facility.
- **narrative** — a plain-English sentence assembled from whichever signals fired (e.g. "Pattern anomaly ... FRP 40 MW ... 4 repeat detections ... <=0.8 km from Oil Refinery / Petrochemical ... Land cover: Industrial Land Use").
- **nearest_city, dist_nearest_city_km, near_population** — from a hard-coded list of 30 Indian cities in risk_engine.py.
- **risk factor breakdown** — `[PLANNED]` additive list of (reason, +points) for the Investigation view.

```
THERMAL DETECTION (scored FIRMS row)
     |
EVIDENCE (only signals that are actually true)
|-- anomaly_flag == 1              (+30)
|-- frp_mw = 40  -> >= 30          (+25)
|-- persistence_count = 4 -> >= 4   (+20)
|-- dist_nearest_facility_km = 0.8  (+20)
|-- predicted_label 'A'             (+10)   [example numbers]
     |
RISK SCORE = 30+25+20+20+10 = 105 -> clamped view 100
     |
SEVERITY = CRITICAL   (>= 65)
     |
ALERT  (inserted into SQLite; CRITICAL/HIGH start at status ALERTED)
```

Alert lifecycle (alert_store.py LIFECYCLE_STATES)

State	What it means	How it is reached
DETECTED	New low-severity alert, not yet acted on	Initial status for LOW
VALIDATING	System/analyst confirming it's real	Initial status for MEDIUM
ALERTED	Raised for attention	Initial status for CRITICAL / HIGH
ESCALATED	Pushed up for field verification	Analyst clicks 'Escalate'
MONITORING	Acknowledged, being watched	Analyst clicks 'Acknowledge' (sets acknowledged_at)
EXTINGUISHED	Closed / resolved	Analyst clicks 'Resolve'

REMEMBER

Initial status is set by severity: CRITICAL/HIGH -> ALERTED, MEDIUM -> VALIDATING, LOW -> DETECTED. Only a human moves it further. The agent [PLANNED] is read-only and cannot.

Part 10 — "Why Was This Flagged?" (Investigation)

`[PLANNED]` as a dedicated page (`queries.get_investigation(alert_id)`); today the same information exists in the alert row + expander of the single-page app. It is **assembled from existing alert fields** — nothing new is invented.

```
DETECTION            acq_date/time, satellite = VIIRS 375m, FIRMS confidence
     |
LOCATION             lat/lon, nearest city + state, land-cover zone
     |
THERMAL EVIDENCE     frp_mw, bt_kelvin, persistence_count, day/night
     |
INDUSTRIAL CONTEXT   dist_nearest_facility_km, hazard_facility_type
     |
MODEL RESULT         predicted_label, prob_A, prob_B_candidate, anomaly_flag
     |
RISK FACTORS         the +points signals that actually fired  [PLANNED breakdown]
     |
RISK SCORE           0-100  ->  SEVERITY band
     |
RECOMMENDED ACTION   derived string, e.g. "ESCALATE FOR FIELD VERIFICATION"
```

Why flagged — checklist rule

Each line shows ONLY if the underlying value is real: repeat detections (persistence >= 2), near industrial facility (distance below threshold), industrial land-use match, elevated FRP (above training median), night-time detection, pattern anomaly. False or unknown signals are omitted — never shown as an empty checkbox, never fabricated.

> **EXAMPLE**
>
> [EXAMPLE — illustrative, not a stored alert] Industrial Fire / Abnormal Thermal Event, near Paradip, Odisha. RISK 78/100, model class probability 0.52, status ALERTED. Detection: FRP 9.5 MW, 4 repeat detections, night pass, VIIRS 375 m. Context: 0.8 km from an Oil Refinery / Petrochemical facility; land cover Industrial Land Use. Why flagged: repeat detections; near industrial facility; industrial land-use match; night-time detection; pattern anomaly. Recommended action: ESCALATE FOR FIELD VERIFICATION — persistent high-intensity anomaly near industrial infrastructure.

> **REMEMBER**
>
> Don't just say WHAT we detected. Explain WHY the system thinks it matters. Explainability is a scoring criterion for SIH judges.

Part 11 — GIS / Geospatial Intelligence

ELI5	Every detection is a pin on a map, and we can measure how far that pin is from anything else.
OUR BUILD	pydeck ScatterplotLayer on a Carto dark basemap (no Mapbox token). Points coloured by class or severity; incident + facility layers; GeoJSON/CSV export. State/region resolver is [PLANNED] (bundled india_states.geojson + pure-Python point-in-polygon).
WHY	Geographic context is how we separate industrial activity from unrelated heat — the same reading means different things in an industrial corridor vs a forest.
MEMORY	**Coordinates + nearest facility + zone = geographic context.**

Concept	In our project
latitude / longitude	Decimal degrees; the primary key of a detection.
Point data	Each detection/alert is a single Point feature.
GeoJSON	Export format: a FeatureCollection of Points, each with the full alert attribute table. actions.export_geojson [PLANNED] / _alerts_to_geojson [IMPLEMENTED in app.py].
State detection	[PLANNED] point-in-polygon against a simplified India state GeoJSON -> 'Odisha', 'Jharkhand', ...
Region detection	[PLANNED] REGIONS map, e.g. 'eastern india' = {Odisha, Jharkhand, West Bengal, Bihar, ...}.
Facility proximity	BallTree (haversine) nearest-neighbour to facilities.parquet -> distance + type.
Map layers	detections (class/severity colour), confirmed incidents, facilities toggle, legend, tooltips.

```
DETECTION
    |
COORDINATES (lat, lon)
    |
NEAREST FACILITY   (BallTree haversine over 72,624 facilities)
    |
FACILITY TYPE + DISTANCE   (e.g. "Oil Refinery / Petrochemical", 0.8 km)
    |
GEOGRAPHIC CONTEXT   (+ land-cover zone, + nearest city, + [PLANNED] state)
    |
MAP   (pydeck marker, colour = class, click -> Investigation [PLANNED])
```

REMEMBER

The map answers exactly one question well: WHERE are the thermal anomalies? Don't drown it in every attribute at once.

Part 12 — Our Facility Data

ELI5	We keep a giant list of industrial places. For every heat detection we ask: which industrial place is closest, and how far?
OUR BUILD	data/processed/facilities.parquet — 72,624 rows (34,936 from WRI Global Power Plant DB + 37,688 from OpenStreetMap landuse=industrial). Columns: facility_id, lat, lon, facility_type, source, name, country. ~39,277 rows are in India.
WHY	Distance-to-facility is our single most important model feature (importance 0.29) and a major risk signal (+20/+12/+6).
MEMORY	**Big list of factories + nearest-neighbour search = industrial context.**

How proximity is computed (the BallTree pattern — it IS used)

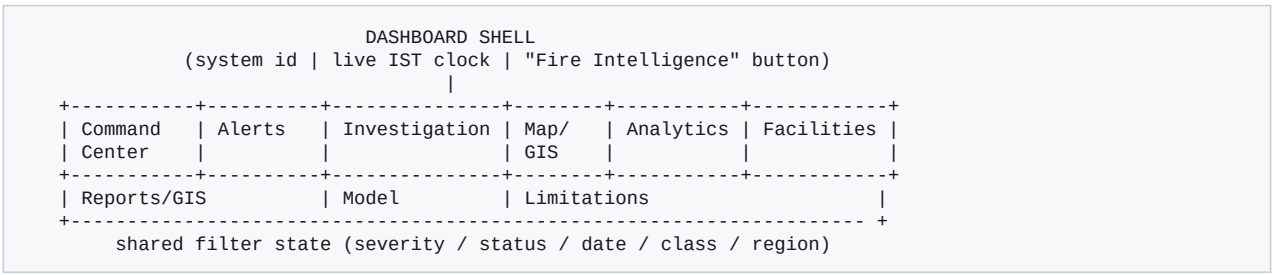
```
# src/features/engineer.py  _query_nearest_facility  (also score_incidents.py)
from sklearn.neighbors import BallTree
coords_rad = np.radians(facilities[["lat","lon"]].values)
tree  = BallTree(coords_rad, metric="haversine")          # build once
dist_rad, idx = tree.query(np.radians(points), k=1)        # nearest for all points
dist_km = dist_rad[:, 0] * 6371.0                          # earth radius
```

A BallTree is a spatial index that makes "nearest of 72k points" fast for every one of hundreds of detections, using great-circle (`haversine`) distance on the sphere.

> **JUDGE QUESTION**
>
> **Q:** How do you know a facility is nearby / that it's the right one?
>
> **A:** We take the single nearest facility from a 72,624-row table (power plants + OSM industrial land use) using a haversine BallTree, and we report its type and distance. It is context, not proof — a hotspot 300 m from a refinery is more suspicious, but proximity alone is never treated as a label.

Part 13 — Dashboard

[IMPLEMENTED] today: one Streamlit page (dashboard/app.py, ~1,760 lines) = situation header + control bar + alert feed + India map + 6 tabs (Timeline, GIS Export, Classification, Incidents, Model, Limitations). [PLANNED]: split into the sections below via st.navigation, same features, clearer layout.



Section	Question it answers	Shows	Operator can	Status
Command Center	What's happening, how bad, where, what needs attention?	Situation counts, live map, top-5 priority alerts, 14-day activity strip	Jump to an investigation; View All Alerts	[PLANNED] (parts exist)
Alerts	Which alerts, in what order?	Full feed, severity-grouped, paginated, filters	Filter; expand; Acknowledge / Escalate / Resolve; View Investigation	[IMPLEMENTED] feed
Investigation	Why does THIS alert matter?	Detection / context / why-flagged / classification / risk factors / recommended action	Acknowledge / Escalate / Resolve; Show on Map	[PLANNED]
Map / GIS	Where are the anomalies?	India pydeck map, class/severity colour, incident + facility layers, legend, tooltips	Zoom, toggle layers, click a detection -> Investigation	[IMPLEMENTED] map
Analytics	How does now compare to before?	Activity strip, calendar, period stats, playback, class/land-cover tables, [PLANNED] baseline FRP	Pick a date/range -> filters map & alerts	[IMPLEMENTED] + [PLANNED] baseline
Facilities	What's happening around known industry?	Facilities with nearby detections: type, state, counts, max risk	Filter map/alerts to a facility	[PLANNED]
Reports / GIS	Get the picture out	GeoJSON + CSV export (filter-aware) + preview; [PLANNED] incident report	Download artefacts	[IMPLEMENTED] export
Model	How does the system work?	Real pipeline diagram, 3-way evaluation, feature importance	Read	[IMPLEMENTED] content
Limitations	What can't it do?	FIRMS resolution, revisit, land-cover, NRT-only, framing	Read	[IMPLEMENTED] content

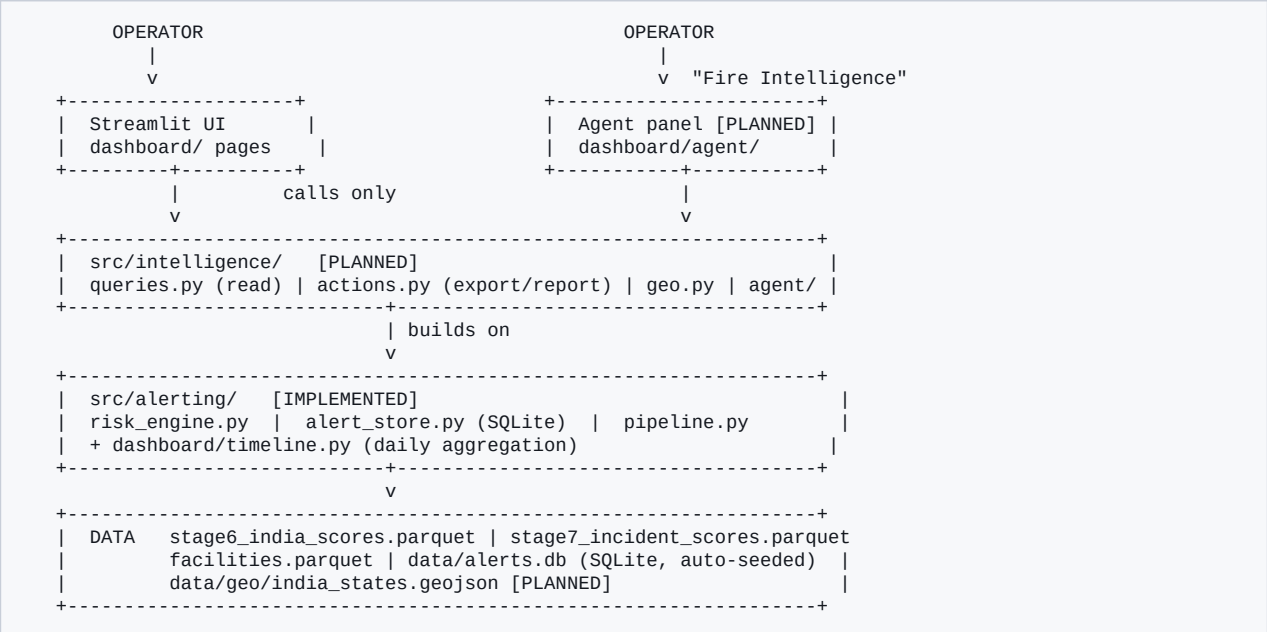
Streamlit — only what you need to know

- Streamlit re-runs the whole script top-to-bottom on every interaction.
- State that must survive a re-run lives in st.session_state (filters, current page, selected alert, agent history). [PLANNED] dashboard/state.py centralises this.
- @st.cache_data stops us re-reading parquet files every re-run.
- The map is pydeck (deck.gl) via st.pydeck_chart.
- On first launch, if data/alerts.db is missing, pipeline.run(fresh=True) seeds it automatically.

REMEMBER

Manual and agent paths both write the SAME session_state filters, so "agent filtered the map" and "I filtered the map" are identical downstream.

Part 14 — Application Architecture



- **UI layer** — draws things, holds no logic. Today one file; `[PLANNED]` multipage.
- **src/intelligence/** `[PLANNED]` — the single place both the UI and the agent get data. Framework-agnostic (no Streamlit import) so a React frontend could be added later untouched.
- **src/alerting/** `[IMPLEMENTED]` — the engines: risk scoring, the SQLite alert store + lifecycle, the seeding pipeline.
- **Data** — committed parquets (model already scored) + a local SQLite DB. The dashboard never loads the ML model file (`stage6_model.joblib` is git-ignored).

> **REMEMBER**
>
> ONE-MINUTE ARCHITECTURE: "An offline pipeline scores NASA FIRMS detections and writes parquet files. A rule-based engine turns those into risk-scored alerts in a small SQLite database with a lifecycle. A Streamlit dashboard reads everything through one service layer, src/intelligence, and shows a map, an alert feed, and per-alert investigations. A read-only Fire Intelligence Agent [PLANNED] sits on the same service layer: it turns a plain-English question into a call to those same functions and pushes the answer back into the dashboard's shared filter state. Nothing needs an internet connection or an API key to run."

Part 15 — Fire Intelligence Agent

[PLANNED]. The **guaranteed baseline is the deterministic offline parser** — no API, no network. Claude is [OPTIONAL].

```
USER QUESTION   "Show me critical industrial fires in Gujarat."
     |
INTENT DETECTION      -> list / filter alerts   (verb: "show me")
     |
ENTITY EXTRACTION     severity  = CRITICAL
     |                output_class = Industrial Fire
     |                location  = Gujarat  (from geo.REGIONS / state list)
     |
TOOL SELECTION        queries.list_alerts(filters=...)
     |
QUERY REAL DATA       src/intelligence -> src/alerting -> alerts.db + parquet
     |
RESULT               list of real alerts (or "none match")
     |
RESPONSE             short NL answer + result cards
     |                [Open Investigation] [Show on Map] [Generate Report]
     |
UI ACTION            ui_action = {nav:"map", filters:{severity:["CRITICAL"],
                     state:"Gujarat", output_class:["Industrial Fire..."]}}
                     -> written to session_state -> st.rerun()
```

Sentence breakdown

Fragment	Becomes
"Show me"	intent = list/filter + navigate
"critical"	filters.severity = ['CRITICAL']
"industrial fires"	filters.output_class = ['Industrial Fire / Abnormal Thermal Event']
"in Gujarat"	filters.state = 'Gujarat'

Parts of the agent

- **deterministic.py** — regex/keyword intent + entity extraction. The always-on baseline.
- **tools.py** — a fixed registry of READ-ONLY tools (1:1 with `queries.py` + export/report). No state-changing tool is registered.
- **queries.py** — the real data functions (list_alerts, rank_alerts, situation_summary, compare_regions, get_investigation, facilities_with_activity, analytics_summary, baseline_comparison, incidents).
- **result cards** — Open Investigation / Show on Map / Generate Report buttons attached to an answer.
- **ui_action** — {nav, filters, focus_alert_id} applied through `state.py`, then `st.rerun()`.
- **shared state** — the agent writes the same filters a human would.
- **read-only security** — see Part 16.

> **[OPTIONAL] Claude integration**
>
> If `ANTHROPIC_API_KEY` is set AND `import anthropic` works, `runtime.py` uses a Claude tool-use loop (model `claude-sonnet-5`) over the *same read-only tool registry*. It can do nothing the deterministic parser cannot.
>
> Any failure (no key, timeout, network, quota) -> silent fallback to the deterministic parser + an "offline mode" note. **The app never depends on Claude.**

REMEMBER

Agent = a natural-language interface to our existing intelligence, NOT a replacement for the intelligence. It calls the same functions the buttons call.

Part 16 — Security / Why the Agent Is Read-Only

The agent CAN	The agent CANNOT
Query alerts / detections / facilities	Modify the database
Filter, rank, aggregate, compare regions	Execute SQL or arbitrary code
Navigate to a section	Run shell commands
Focus / filter the existing map	Change an alert's status (Acknowledge / Escalate / Resolve)
Open an investigation	Delete or create alerts
Generate a GeoJSON / CSV / incident report	Re-run the pipeline or retrain the model

- The LLM (or parser) can only call **named tools with typed arguments** from a fixed registry — there is no free-form query surface.
- No state-changing tool is registered this round. `actions.set_alert_status` exists but is called **only by the manual UI buttons**.
- Worst case of a bad/hallucinated agent request: a wrong-but-harmless list or filter. Nothing is written.
- Only the `[OPTIONAL]` Claude path makes any network call, and it sends only the question text + tool schemas/results — never credentials or bulk data.

> **REMEMBER**
>
> Read-only = the agent can help you look, never touch. Consequential actions stay human. This is a deliberate safety decision for the internal hackathon.

Part 17 — End-to-End Example

One detection, traced through the whole system. Field names are real; the specific values are an **illustrative example**.

```
SATELLITE      Suomi-NPP / NOAA-20 VIIRS overpass at night sees a hot 375 m pixel
     |
NASA FIRMS     row: latitude=20.31, longitude=86.61, bright_ti4=331, frp=9.5,
               acq_date=2026-08-26, daynight=N, confidence=nominal
     |
INGEST/CLEAN   src/ingestion/firms.py -> tagged split=india_holdout; tidy row
     |
FEATURES       bt_kelvin=331, frp_mw=9.5, persistence_count=4,
(engineer.py)  dist_nearest_facility_km=0.8, agri_season_flag=0,
               day_night_bin=0, acq_month=8
     |
FACILITY CTX   nearest_facility_type="refinery" -> hazard "Oil Refinery /
               Petrochemical"; land_cover_context="Industrial Land Use"
     |
ML (RF)        predict_proba -> prob_A=0.52, prob_B_candidate=0.48
               predicted_label="A"
     |
ANOMALY        max(prob)=0.52 < 0.55  ->  anomaly_flag = 1
     |
OUTPUT CLASS   anomaly_flag==1  ->  "Industrial Fire / Abnormal Thermal Event"
     |
RISK ENGINE    +30 anomaly, +8 FRP(>=5), +20 persistence(>=4), +20 facility(<1),
               +10 pred=='A', +5 night, +10 near city  = 103 -> shown 100
     |
SEVERITY       >= 65  ->  CRITICAL
     |
ALERT STORE    alert_store.insert_alerts -> new row in data/alerts.db,
               status = ALERTED (because CRITICAL), narrative auto-written
     |
DASHBOARD      appears top of the Alerts feed (sorted by risk_score DESC),
               red marker on the India map
     |
INVESTIGATION  [PLANNED page] why-flagged: repeat detections / near industrial
               facility / industrial land-use / night-time / pattern anomaly;
               recommended action: ESCALATE FOR FIELD VERIFICATION
     |
AGENT [PLANNED] "highest-risk industrial fire in Odisha?" -> queries.rank_alerts
               -> returns THIS alert -> [Open Investigation] [Show on Map]
```

REMEMBER

If you can narrate this one chain end to end, you can present the whole project.

Part 18 — What Is Actually Implemented?

Tag	Meaning
`[IMPLEMENTED]`	Works today in the repo you can run right now.
`[PLANNED]`	In the approved plan / architecture.md, not yet coded.
`[OPTIONAL]`	Works only if an extra thing (e.g. an API key) is present; not required.
`[NOT SUPPORTED]`	Deliberately excluded. Must never be claimed.

Component	Status	What it actually does
FIRMS + VNF + facility ingestion (Stages 1-2)	[IMPLEMENTED]	Downloads and tags data; builds facilities.parquet (72,624 rows).
Feature engineering (Stage 4)	[IMPLEMENTED]	features_stage4.parquet, ~419k rows, 25 cols.
Spatial split + leakage checks (Stage 5)	[IMPLEMENTED]	1-degree grid 80/20 split; assertions block India in train/val.
Random Forest train + 3-way eval (Stage 6)	[IMPLEMENTED]	Model + stage6_india_scores.parquet + reports/stage6_evaluation.txt.
Incident scoring (Stage 7)	[IMPLEMENTED]	30 incidents scored; 21/30 anomaly-flagged; stage7_incident_scores.parquet.
Risk engine	[IMPLEMENTED]	risk_engine.py: output_class, risk_score, severity, land_cover, hazard type, narrative, city context.
Alert store + lifecycle (SQLite)	[IMPLEMENTED]	alert_store.py: insert/get/update_status/counts; 6 lifecycle states.
Seeding pipeline	[IMPLEMENTED]	pipeline.run(fresh=) seeds alerts.db from scores; auto-runs on first launch.
Single-page Streamlit dashboard	[IMPLEMENTED]	app.py: header, control bar, alert feed, India map, 6 tabs.
India map (pydeck + Carto dark)	[IMPLEMENTED]	class/severity colour, incident overlay, facility toggle, tooltips.
Historical timeline / calendar / playback	[IMPLEMENTED]	timeline.py + Timeline tab; daily severity aggregation over alerts.db.
GIS export (GeoJSON + CSV)	[IMPLEMENTED]	GIS Export tab; Point features with full attribute table.
Manual Acknowledge / Escalate / Resolve	[IMPLEMENTED]	Buttons in the alert expander -> alert_store.update_status.
Model + Limitations transparency panels	[IMPLEMENTED]	Tabs describing the real pipeline and caveats.
src/intelligence/ service + tool layer	[PLANNED]	queries.py / actions.py / geo.py — one place UI + agent get data.
Multipage IA (Command Center, Investigation, Facilities, Reports pages)	[PLANNED]	Reorganise the same features via st.navigation.
lat/lon -> Indian state / region resolver	[PLANNED]	geo.py + bundled india_states.geojson, pure-Python point-in-polygon.
Investigation view (assembled)	[PLANNED]	get_investigation(alert_id): detection/context/why/risk/action.
Facilities activity view	[PLANNED]	facilities_with_activity: facilities with nearby detections + counts.
Baseline-vs-current FRP comparison	[PLANNED]	baseline_comparison(); shows 'insufficient history' when data is thin.
Risk factor breakdown in RiskResult	[PLANNED]	Additive (reason,+points) list for the Investigation risk section.

Component	Status	What it actually does
Fire Intelligence Agent (deterministic, read-only)	[PLANNED]	NL query -> tool call -> real data -> answer + result cards + ui_action.
Claude API agent runtime	[OPTIONAL]	Used only if ANTHROPIC_API_KEY set; same read-only tools; silent fallback.
Agent Acknowledge / Escalate / Resolve	[NOT SUPPORTED]	Deliberately deferred. Agent never changes incident state this round.
Confirmed-industrial-fire detection claim	[NOT SUPPORTED]	No such dataset exists; we detect anomalous departures, not confirmed fires.
Historical FIRMS archive / GIHS / land-cover raster	[NOT SUPPORTED]	Not integrated; land cover is a coordinate-zone heuristic.
Real-time streaming / auth / notifications	[NOT SUPPORTED]	Out of scope; FIRMS NRT batch + single-user local app.

Part 19 — Limitations (be honest)

Limitation	Why it exists	How to explain it to judges
No 'confirmed fire' claim	No dataset of confirmed industrial fires exists anywhere (checked: PESO, NDMA, NCRB, DGMS, CPCB, no global equivalent).	"We detect anomalous departures from known persistent-industrial and natural-fire patterns — a screening tool for human verification, not a fire confirmation system."
FIRMS resolution	VIIRS pixels are 375 m, MODIS 1 km — small or short-lived events can be missed or merged.	"We inherit the sensor's spatial limits; our value is the interpretation layer on top."
Satellite revisit / clouds	Polar-orbiting satellites pass a few times a day; cloud blocks the IR view.	"Coverage is periodic, not continuous — persistence is measured over available passes."
NRT only (~5 days)	FIRMS NRT API returns only the last ~5 days; no historical archive is wired in.	"Our timeline depth and 2019-2023 incident date-matching are limited; the archive is a known next step."
Proxy labels	'A' = near a VIIRS Nightfire flare site; 'B_candidate' = everything else global. Not human-verified.	"Labels are a defensible proxy from an authoritative flare catalogue, and we say so openly."
Thin Class A / low F1	~1,900 positive training rows -> spatial-holdout Class A F1 = 0.18.	"Minority-class recall is limited by label scarcity; a historical archive or GIHS would improve it."
Land cover is a heuristic	No MODIS/ESA land-cover raster integrated; we use lat-lon zone rules.	"A pragmatic approximation; a real raster is a drop-in improvement."
Baseline comparison often empty	Needs many historical days; NRT gives ~5.	"We show it only when the data supports it, otherwise we say 'insufficient history' rather than fake a number."
Incident thermal features imputed	No historical FIRMS for 2019-2023, so bt/frp/persistence/day-night are NaN -> median-imputed.	"Incident scores are geographic-context baselines, not event classifications."
Agent is read-only	Deliberate safety choice for the hackathon.	"The agent helps you look; a human still makes every state change."
Offline / single machine	By design — must run with no network, no keys.	"Reliability for a live demo beats a cloud dependency."

Part 20 — Judge Questions (FAQ)

Problem

Q: What problem does SIH26162 solve?

Short: Satellites see heat but not cause; we classify industrial vs natural thermal events over India and deliver it as GIS.

Deeper: The raw FIRMS feed can't tell a refinery flare from a refinery fire from crop burning. We add context, classification, risk and a map so an operator can prioritise and investigate.

Q: Who is this for?

Short: Disaster-management and industrial-safety monitoring analysts.

Deeper: Also environmental-compliance teams and technical evaluators. The dashboard is designed as an operations tool, not a report.

Q: Why is industrial context central?

Short: The meaning of a hotspot depends on what's near it.

Deeper: Distance-to-facility is our top model feature (0.29) and a major risk signal; it's how we separate industrial relevance from a forest fire.

Data

Q: Where does your data come from?

Short: NASA FIRMS (thermal), VIIRS Nightfire (flare labels), WRI GPPD + OpenStreetMap (facilities).

Deeper: FIRMS NRT for India + 6 non-India training regions; VNF as a labelling oracle; 72,624 facilities; 30 curated incidents for independent evaluation.

Q: Can this work in real time?

Short: Near-real-time — FIRMS NRT updates every few hours; we run in batches.

Deeper: The pipeline is batch today. Streaming ingestion is out of scope but the architecture (score -> parquet -> alert store) would extend to it.

Q: Why hold India out of training?

Short: So the model learns physics, not Indian locations.

Deeper: Global training + India geographic holdout tests whether learned thermal patterns generalise to India — a stronger claim than memorising Indian facilities. India is still used for context, evaluation and deployment.

Q: What if there's no historical data?

Short: We degrade honestly — features are imputed and we say so.

Deeper: FIRMS NRT is ~5 days. For the 2019-2023 incidents, thermal features are NaN -> median-imputed, so those scores are geographic-context baselines, not event classifications. The timeline shows only what's in alerts.db.

ML

Q: Why Random Forest, not deep learning?

Short: 7 tabular features, tiny positive class, explainability required.

Deeper: Trees need no scaling, handle imbalance with class weights, expose feature importance, and don't overfit ~1,900 positives the way a CNN would. A CNN also needs image inputs we don't have.

Q: Why not a CNN on satellite imagery?

Short: We don't ingest imagery, and there's no labelled image dataset of industrial fires.

Deeper: FIRMS gives tabular detections. Building a CNN would mean sourcing and labelling raster scenes — infeasible and unnecessary for 7 features.

Q: What are your labels?

Short: A = FIRMS row within 5 km of a VIIRS Nightfire flare site; B_candidate = other global FIRMS.

Deeper: Proxy labels from an authoritative flare catalogue. No confirmed-incident labels are used for training — none exist.

Q: How accurate is it?

Short: ~98% overall accuracy, but the honest metric is Class A F1 = 0.18 on spatial holdout.

Deeper: Overall accuracy is inflated by the easy majority class. We report all three evaluations and the leakage gap openly.

Q: What does anomaly_flag mean?

Short: The forest couldn't reach 55% confidence for either class.

Deeper: max(prob_A, prob_B_candidate) < 0.55. Those rows become 'Industrial Fire / Abnormal Thermal Event' — a human-review candidate, not a conclusion.

Q: How do you prevent data leakage?

Short: Split by 1-degree spatial grid cells, not randomly; India fully held out; assertions in code.

Deeper: Repeated detections from one site can't span train/val. check_no_india_in_training and check_no_grid_overlap raise LeakageError if violated.

Risk

Q: How is risk calculated?

Short: A transparent additive rule engine, 0-100, then banded into severity.

Deeper: Points for anomaly (+30), FRP, persistence, facility proximity, predicted class, FIRMS confidence, night, city proximity. >=65 CRITICAL, >=40 HIGH, >=20 MEDIUM, else LOW. All in risk_engine.py.

Q: Is risk the same as the ML output?

Short: No. ML says what it is; risk says how concerning it is.

Deeper: A correctly-classified routine flare can be LOW risk; an anomaly near a chemical plant is CRITICAL. Different code, different purpose.

Q: Why a rule engine and not ML for risk?

Short: Transparency and control — every point is explainable to a judge and an operator.

Deeper: We can show exactly which signals drove a score. There's also no labelled 'risk' target to learn from.

GIS

Q: How do you know a facility is nearby?

Short: Nearest-neighbour search over 72,624 facilities with a haversine BallTree.

Deeper: We report the single closest facility's type and great-circle distance. Context, never a label.

Q: How do you distinguish industrial fire from wildfire?

Short: Persistence + industrial proximity + land-cover zone + night behaviour + the RF pattern.

Deeper: A wildfire is bursty, often in forest/cropland, far from industry; a persistent source re-lights every pass near a facility. The model weighs these; the risk engine and land-cover context reinforce it.

Q: What is GeoJSON and why export it?

Short: A standard geographic JSON; each alert is a Point with attributes.

Deeper: So analysts can open our alerts in QGIS/ArcGIS. That's PS deliverable (ii): a GIS-based solution.

Dashboard

Q: Walk me through the operator workflow.

Short: Command Center -> Alerts -> Investigation -> act; Map and Analytics for context.

Deeper: DETECT -> CLASSIFY -> VALIDATE -> PRIORITIZE -> EXPLAIN -> ACT. Filters are shared across sections; clicking a map marker opens the investigation.

Q: Is the UI just a prototype?

Short: It's a working Streamlit app today; the multipage reorg is planned.

Deeper: Single-page app with a live map, alert feed, timeline, GIS export and manual actions all working. The plan splits it into clearer sections without removing features.

Agent

Q: Why do you need the agent?

Short: To ask the platform instead of driving many filters by hand.

Deeper: It's a second interface over the same intelligence layer — a judge can ask 'highest-risk persistent sources near industry in eastern India, last 7 days' and get ranked, explained results with one-click navigation.

Q: Why is the agent offline / does it need an API key?

Short: No. The deterministic parser is the guaranteed baseline; Claude is optional.

Deeper: Everything runs with no network and no key. If ANTHROPIC_API_KEY is present the Claude runtime is used over the same read-only tools, with silent fallback on any failure.

Q: Can the agent change alerts?

Short: No — it is strictly read-only.

Deeper: It can query, filter, navigate, focus the map, open investigations and generate reports. It cannot Acknowledge/Escalate/Resolve, run SQL, or touch the database. Consequential actions stay human.

Q: Is the agent just a chatbot wrapper?

Short: No — it calls the exact functions the dashboard buttons call.

Deeper: Natural language -> intent + entities -> a named read-only tool in src/intelligence -> real data -> answer + result cards that drive the shared UI state.

Security / Validation

Q: What stops the agent doing something dangerous?

Short: It can only call a fixed registry of read-only tools with typed arguments.

Deeper: No free-form query surface, no SQL/shell/eval, no state-changing tool registered. Worst case is a wrong list, never a write.

Q: How do you validate the whole system?

Short: Unit tests for ingestion/features/split today; planned tests for the service + agent layer; the 3-way model evaluation.

Deeper: tests/ has 49 passing tests for leakage and features. Planned: geo, queries, actions, deterministic-parser tests. Model: random vs spatial vs India holdout.

Limitations / Future

Q: Biggest weakness?

Short: Label scarcity for the industrial class (Class A F1 = 0.18).

Deeper: Only ~1,900 positive examples via the VNF oracle. A historical FIRMS archive or the GIHS dataset would materially help recall.

Q: What would you add next?

Short: Historical FIRMS archive, then GIHS + a land-cover raster, then agent actions with confirmation.

Deeper: Archive unlocks real temporal incident matching and a deeper baseline. GIHS + raster improve classification. Agent write-actions are a deliberate later step behind a confirmation gate.

Q: Could this deploy for real?

Short: The intelligence layer could; it's built to be frontend-agnostic.

Deeper: src/intelligence has no UI dependency, so a FastAPI + React front end could sit on it. Real deployment also needs the archive, auth and streaming.

Part 21 — 30-second / 1-minute / 3-minute Explanations

30 seconds (problem + solution)

EXAMPLE

"NASA's FIRMS satellites detect heat anywhere on Earth but can't say what's burning. Over an industrial region that matters — a refinery flare and a refinery fire look the same from orbit. We enrich every detection with industrial and geographic context, classify it with a Random Forest trained on global data with India held out, flag the uncertain ones as anomalies, score the risk, and present prioritised, explained alerts on a GIS dashboard."

1 minute (+ architecture + differentiation)

EXAMPLE

"[30-second version] ... Under the hood: an offline pipeline scores FIRMS detections and writes parquet files; a transparent rule engine turns those into risk-scored alerts in a SQLite store with a full lifecycle; a Streamlit dashboard shows a live India map, an alert feed, and per-alert investigations that explain WHY each alert was flagged. What makes it different: we're honest that no confirmed-fire dataset exists, so we frame it as anomaly screening; we prevent spatial leakage and hold India out entirely; and there's an optional read-only natural-language agent over the same intelligence layer. It all runs offline with no API key."

3 minutes (full technical story)

EXAMPLE

"The problem statement asks us to separate industrial fires and persistent industrial thermal sources from natural and agricultural fires, as a GIS solution. The catch, which we researched first, is that no dataset of confirmed industrial fires exists anywhere, and a FIRMS hotspot carries no cause label. So we don't claim fire detection.

Data: FIRMS near-real-time detections for India plus six non-India regions for training; the VIIRS Nightfire flare catalogue as a labelling oracle; 72,624 industrial facilities from WRI and OpenStreetMap. We label a global FIRMS row 'A' (persistent industrial source) if it's within 5 km of a known flare site, else 'B_candidate' (natural/agricultural). We engineer 7 features: brightness temperature, FRP, persistence count, distance to nearest facility, agri-season, day/night, month.

Model: a Random Forest, 300 trees, class-balanced, median imputation. We evaluate three ways — a random split (0.97, inflated), a spatial-grid holdout (0.98 overall but Class A F1 only 0.18, the honest number), and the India geographic holdout where 8.4% of detections are flagged as anomalies. If the forest can't reach 55% confidence for either class, we flag the detection as an anomaly for human review.

Then a separate rule-based risk engine scores 0-100 from anomaly status, FRP, persistence, facility proximity, night, and city proximity, and bands it CRITICAL/HIGH/MEDIUM/LOW. Alerts go into a SQLite store with a six-state lifecycle. The Streamlit dashboard shows a situation overview, an India map, the alert feed, a historical timeline, GIS export, and — planned — a dedicated Investigation view that lists exactly which signals fired. An optional read-only Fire Intelligence Agent lets you ask all of this in plain English; it calls the same functions the buttons do and can never change state. Everything runs offline."

Part 22 — Memory Sheets

Cheat Sheet 1 — Whole project in 10 lines

```
1  Goal: separate industrial fires / persistent industrial sources from natural fires; deliver as GIS.
2  Data: NASA FIRMS NRT (India + 6 global regions), VIIRS Nightfire, 72,624 facilities (WRI+OSM).
3  Labels: proxy. A = FIRMS within 5km of a VNF flare site; B_candidate = other global FIRMS.
4  Features (7): bt_kelvin, frp_mw, persistence_count, dist_nearest_facility_km, agri_season_flag,
   day_night_bin, acq_month.
5  Model: RandomForest 300 trees, class_weight=balanced, median impute. Predicts A vs B_candidate.
6  Anomaly: max(prob) < 0.55  ->  anomaly_flag = 1  ->  "Industrial Fire / Abnormal Thermal Event".
7  Eval: random 0.9725 (inflated) | spatial holdout 0.9806, Class A F1 0.18 (honest) | India 8.4% anomaly.
8  Risk engine (rules, 0-100): anomaly+30, FRP, persistence, facility dist, night, city -> severity band.
9  Alerts: SQLite, lifecycle DETECTED->VALIDATING->ALERTED->ESCALATED->MONITORING->EXTINGUISHED.
10 UI: Streamlit + pydeck India map + alert feed + timeline + GIS export. Agent [PLANNED], read-only, offline.
```

Cheat Sheet 2 — Data pipeline

```
ACQUIRE  -> INGEST(tag split) -> CLEAN -> FEATURES(7) -> FACILITY CTX(BallTree)
-> CLASSIFY(RF) -> ANOMALY(0.55) -> RISK(rules) -> ALERT(SQLite)
Files: src/ingestion/* , src/features/engineer.py , src/model/{assemble,train,split}.py ,
       src/alerting/{risk_engine,alert_store,pipeline}.py
Outputs committed: stage6_india_scores.parquet, stage7_incident_scores.parquet, facilities.parquet
NOT in repo: stage6_model.joblib, stage5_*.parquet, data/raw/ (all git-ignored)
```

Cheat Sheet 3 — ML

```
Target : 2 classes  A (persistent industrial)  vs  B_candidate (natural/agri)
Labels : VNF spatial oracle, 5 km radius. No confirmed-fire labels.
Samples: train 270,238 (A=1,655) | val 64,864 (A=246) | India holdout 705 (no labels)
Model  : Pipeline(SimpleImputer(median), RandomForestClassifier(
         n_estimators=300, min_samples_leaf=10, class_weight='balanced', random_state=42))
Outputs: predicted_label, prob_A, prob_B_candidate, anomaly_flag(=max(prob)<0.55)
Feat.imp: dist_facility .29 | day_night .25 | bt_kelvin .21 | persistence .14 | frp .10 | month 0 | agri 0
Never claims: "confirmed industrial fire"
```

Cheat Sheet 4 — Risk

```
score = 0
 +30 anomaly_flag==1
 +25/+15/+8   frp_mw >=30 / >=15 / >=5
 +20/+10      persistence_count >=4 / >=2
 +20/+12/+6   dist_nearest_facility_km <1 / <5 / <15
 +10 predicted_label=='A'
 +8  FIRMS confidence high
 +5  night
 +10 nearest city <30 km
severity: >=65 CRITICAL | >=40 HIGH | >=20 MEDIUM | <20 LOW
also outputs: land_cover_context, hazard_facility_type, narrative, nearest_city
```

Cheat Sheet 5 — GIS

```
Point data (lat, lon) per detection/alert.
Nearest facility: BallTree(metric='haversine') over 72,624 rows -> type + distance_km.
Map: pydeck ScatterplotLayer, Carto dark basemap, colour by class or severity.
Export: GeoJSON FeatureCollection (Point + full attributes) + CSV. = PS deliverable (ii).
[PLANNED] state/region: point-in-polygon vs bundled india_states.geojson; 'eastern india' region set.
```

Cheat Sheet 6 — Dashboard

```
Today: 1 Streamlit page (dashboard/app.py) = header + control bar + alert feed + India map
       + tabs {Timeline, GIS Export, Classification, Incidents, Model, Limitations}
[PLANNED] pages: Command Center | Alerts | Investigation | Map/GIS | Analytics |
                 Facilities | Reports/GIS | Model | Limitations   (via st.navigation)
```

```
Shared session_state filters: severity / status / date / class / region
Manual actions (IMPLEMENTED): Acknowledge->MONITORING, Escalate->ESCALATED, Resolve->EXTINGUISHED
First run auto-seeds data/alerts.db via pipeline.run(fresh=True)
```

Cheat Sheet 7 — Agent [PLANNED]

```
question -> intent + entities (severity/class/region/timeframe/rank)
        -> named READ-ONLY tool in src/intelligence/agent/tools.py
        -> queries.py (real data) -> answer + result cards + ui_action
Result cards: [Open Investigation] [Show on Map] [Generate Report]
ui_action {nav, filters, focus_alert_id} -> session_state -> st.rerun()
Baseline: deterministic.py (offline, no key). [OPTIONAL] claude.py (claude-sonnet-5), silent fallback.
Read-only: no DB write, no SQL, no shell, no status change.
```

Cheat Sheet 8 — Judge one-liners

```
"Is this a confirmed fire?"      -> No. Anomalous departure from known patterns; needs human check.
"Why Random Forest?"             -> 7 tabular features, tiny positive class, explainability.
"What are your labels?"          -> Proxy: near a VIIRS Nightfire flare site = A; else B_candidate.
"How accurate?"                  -> ~98% overall (inflated); honest Class A F1 = 0.18 spatial holdout.
"Model uncertain?"               -> max(prob)<0.55 -> anomaly_flag -> human review.
"How is risk different?"         -> ML = what it is; risk engine = how concerning (rules, 0-100).
"Nearest facility?"              -> haversine BallTree over 72,624 facilities.
"Does the agent need internet?" -> No. Deterministic offline parser is the baseline.
"Can the agent change alerts?"  -> No. Strictly read-only.
"Real time?"                     -> Near-real-time FIRMS, batch pipeline.
```

Cheat Sheet 9 — Numbers, files, modules

Thing	Value / path
India bbox	`lat 6.0-37.0, lon 68.0-97.5 (src/model/split.py)`
Facilities	`72,624 (34,936 WRI GPPD + 37,688 OSM); ~39,277 in India`
Train / Val / India rows	`270,238 / 64,864 / 705`
Class A training rows	`1,655 train + 246 val = 1,901 (~0.57%)`
RF hyperparams	`n_estimators=300, min_samples_leaf=10, class_weight='balanced', random_state=42`
Anomaly threshold	`0.55 (train.py:59, score_incidents.py:46)`
VNF oracle radius	`5.0 km (assemble.py:57)`
Severity cutoffs	`65 / 40 / 20`
Incidents scored	`30; anomaly-flagged 21 (70%)`
India predictions	`A=309, B_candidate=396, anomaly 59 (8.4%)`
Model file	`data/processed/stage6_model.joblib — GIT-IGNORED, not in repo`
Dashboard entry	`dashboard/app.py (streamlit run dashboard/app.py)`
Alert DB	`data/alerts.db (SQLite, auto-seeded, git-ignored)`
Risk engine	`src/alerting/risk_engine.py`
Alert store	`src/alerting/alert_store.py`
Docs	`context.md, architecture.md, workflow.md, design_brief.md, modeltrain.md`

Part 23 — Master Memory Palace

Icon / role	Analogy	Technical reality (do not distort this)
Eyes	SATELLITE — sees heat	NASA FIRMS VIIRS/MODIS thermal-anomaly detections (lat, lon, FRP, BT, time)
Brain	ML — decides what it is	RandomForest -> predicted_label (A / B_candidate) + probabilities + anomaly_flag
Threat board	RISK ENGINE — how worried to be	rule-based additive score 0-100 -> severity band (separate from ML)
Map	GIS — where + what's nearby	pydeck India map, haversine nearest-facility, land-cover zone, GeoJSON export
Notebook	ALERT MEMORY — what we've seen	SQLite alerts.db with a 6-state lifecycle
Control room	DASHBOARD — where the operator works	Streamlit app: situation view, feed, map, timeline, investigation
Assistant	AGENT — answers questions	[PLANNED] read-only NL interface over the same src/intelligence functions

```
EYES  (satellite)            NASA FIRMS thermal detection
     |                             |
BRAIN (ML)                   RandomForest -> label + prob + anomaly_flag
     |                             |
THREAT BOARD (risk)          rule engine 0-100 -> CRITICAL/HIGH/MEDIUM/LOW
     |                             |
MAP  (GIS)                   coordinates + nearest facility + land-cover + map
     |                             |
NOTEBOOK (alert memory)      SQLite alerts.db + lifecycle
     |                             |
CONTROL ROOM (dashboard)     Streamlit: feed + map + timeline + investigation
     |                             |
ASSISTANT (agent) [PLANNED]  NL question -> read-only tools -> same data
```

REMEMBER

Eyes -> Brain -> Threat board -> Map -> Notebook -> Control room -> Assistant. Seven links. If you can say the technical name for each icon, you know the system.

Part 24 — Final Self-Test (50 questions)

Answers start on the next page. Cover them first.

1. [L1] State the project goal in one sentence.

2. [L1] What does NASA FIRMS give you, and what does it NOT give you?

3. [L1] Name the three output classes shown on the dashboard.

4. [L1] What is the difference between classification and risk?

5. [L1] What does 'GIS solution' mean for PS deliverable (ii)?

6. [L1] Which two data sources give the facility layer?

7. [L1] What runs when you first launch the dashboard with no alerts.db?

8. [L1] Does the project need an internet connection or API key to run?

9. [L1] Name the 6 alert lifecycle states in order.

10. [L1] What is the one-sentence 'master memory' chain?

11. [L2] List the 7 model features.

12. [L2] How are the training labels ('A' / 'B_candidate') created?

13. [L2] How many train / val / India-holdout rows are there?

14. [L2] What model and hyperparameters are used?

15. [L2] Define anomaly_flag precisely.

16. [L2] List the risk-engine point rules from memory.

17. [L2] What are the severity cutoffs?

18. [L2] How is nearest-facility distance computed?

19. [L2] What does the risk engine output besides the score?

20. [L2] Which fields does the Investigation view assemble, in order?

21. [L2] What does the [PLANNED] state/region resolver do and how?

22. [L2] What are the agent's result cards and ui_action?

23. [L2] Where does the dashboard get its data from (which layer)?

24. [L2] What is committed to the repo vs git-ignored, data-wise?

25. [L2] What are the three model evaluations and their headline numbers?

26. [L3] Why Random Forest instead of a neural network?

27. [L3] Why hold India out of training entirely?

28. [L3] Why split by spatial grid instead of randomly?

29. [L3] Why is overall accuracy (0.98) misleading here?

30. [L3] Why is facility proximity a feature but never a label?

31. [L3] Why is risk a rule engine and not another ML model?

32. [L3] Why is the agent read-only for this round?

33. [L3] Why is the deterministic parser the baseline and Claude only optional?

34. [L3] Why does src/intelligence have no Streamlit import?

35. [L3] Why can't we claim 'confirmed industrial fire'?

36. [L4] A judge says '98% accuracy is great!' — correct them.

37. [L4] A judge asks 'so you can detect industrial fires?' — respond precisely.

38. [L4] 'Your Class A F1 is only 0.18, is the model useless?' — defend it.

39. [L4] 'Isn't near-a-factory basically your label?' — rebut.

40. [L4] 'Why not just use the FIRMS confidence field?' — answer.

41. [L4] 'What if the agent hallucinates and escalates a wrong alert?' — answer.

42. [L4] 'How is this different from an existing wildfire dashboard?' — answer.

43. [L4] 'Can this run at national scale in real time?' — answer honestly.

44. [L4] 'Why 0.55 and not 0.5 or 0.6 for the anomaly threshold?' — answer.

45. [L4] 'Where do the 30 incidents fit — are they training data?' — answer.

46. [L5] alerts.db is deleted mid-demo. What happens on the next page load, and which module handles it?

47. [L5] A new FIRMS row has frp=NaN. Trace how it still gets a prediction and a risk score.

48. [L5] The agent answers 'critical alerts in Odisha' but the Map doesn't change. Where are the 3 likely break points?

49. [L5] You must add a 'distance to coastline' feature. Which files change, and what must you re-run?

50. [L5] A teammate imports src.alerting directly inside a new dashboard page. Why is that a problem per the architecture, and the fix?

Self-Test — Answers

1. Take NASA FIRMS heat detections over India, add industrial + geographic context, classify each as persistent industrial source vs natural fire (flag neither-fits as anomaly), risk-score it, and raise prioritised explained alerts on a GIS dashboard.

2. Gives: lat/lon, brightness temperature, FRP, date/time, day-night, confidence. Does NOT give: the cause, a confirmed-fire label, industrial-vs-natural, history older than ~5 days, sub-pixel detail.

3. Industrial Fire / Abnormal Thermal Event; Persistent Industrial Thermal Source; Forest / Agricultural Fire.

4. Classification (ML) = WHAT the thermal event is. Risk (rule engine) = HOW CONCERNING it is. Different code, different logic.

5. Store data and show it as map overlays with exportable geographic formats (GeoJSON/CSV) usable in QGIS/ArcGIS.

6. WRI Global Power Plant Database (34,936) + OpenStreetMap landuse=industrial (37,688).

7. pipeline.run(fresh=True) auto-seeds data/alerts.db from stage6_india_scores.parquet (guard at the top of dashboard/app.py).

8. No. The whole app + deterministic agent run offline with no key. Claude is optional.

9. DETECTED -> VALIDATING -> ALERTED -> ESCALATED -> MONITORING -> EXTINGUISHED.

10. Satellite detects heat -> we collect/process -> add environmental+industrial context -> ML estimates what it is -> risk engine estimates how concerning -> GIS says where+what's nearby -> alert generated -> operator investigates -> agent helps ask questions.

11. bt_kelvin, frp_mw, persistence_count, dist_nearest_facility_km, agri_season_flag, day_night_bin, acq_month.

12. VNF spatial oracle: a global FIRMS row within 5 km of a known VIIRS Nightfire gas-flare site -> 'A'; every other global FIRMS row -> 'B_candidate'. VNF rows themselves are excluded from training.

13. Train 270,238 | Val 64,864 | India holdout 705 (no labels).

14. sklearn Pipeline: SimpleImputer(strategy='median') then RandomForestClassifier(n_estimators=300, min_samples_leaf=10, class_weight='balanced', n_jobs=-1, random_state=42).

15. anomaly_flag = int(max(prob_A, prob_B_candidate) < 0.55). Applied after the forest votes.

16. +30 anomaly; +25/+15/+8 FRP>=30/15/5; +20/+10 persistence>=4/2; +20/+12/+6 facility<1/5/15 km; +10 predicted 'A'; +8 FIRMS confidence high; +5 night; +10 nearest city <30 km.

17. CRITICAL >=65, HIGH >=40, MEDIUM >=20, LOW <20.

18. BallTree(metric='haversine') built over facility lat/lon (radians); query k=1 for every point; distance_rad * 6371 km.

19. output_class, severity, land_cover_context, hazard_facility_type, narrative, nearest_city, dist_nearest_city_km, near_population (and [PLANNED] a risk-factor breakdown).

20. Incident header, Detection, Context, Why Flagged, Classification, Risk Assessment, Recommended Action.

21. [PLANNED] geo.py: point-in-polygon (pure Python ray-casting) against a bundled simplified india_states.geojson -> state name; a REGIONS map groups states (e.g. 'eastern india').

22. Result cards: Open Investigation / Show on Map / Generate Report. ui_action = {nav, filters, focus_alert_id}, applied via state.py then st.rerun().

23. [PLANNED] src/intelligence/ (queries.py / actions.py / geo.py) — the single service layer; the dashboard imports only that, never src.alerting directly.

24. Committed: stage6_india_scores.parquet, stage7_incident_scores.parquet, facilities.parquet, the CSVs/JSONs, [PLANNED] india_states.geojson. Git-ignored: stage6_model.joblib, stage5_*.parquet, data/raw/, data/alerts.db.

25. Random split: accuracy 0.9725 (inflated). Spatial holdout: 0.9806 overall, Class A F1 0.18 (honest). India holdout: no labels, predicted A=309 / B_candidate=396, anomaly 59/705 = 8.4%.

26. 7 tabular features, ~0.6% positive class, and explainability is required. Trees need no scaling, handle imbalance via class weights, expose feature importance. A CNN needs image inputs and far more labels than exist.

27. So the model learns physical/thermal patterns, not Indian locations/facilities. It becomes a geographic-generalisation test. India is still used for context, evaluation and deployment.

28. Repeated detections from the same site/cell would leak across a random split and inflate accuracy. Whole 1-degree grid cells go entirely to train or val.

29. It is dominated by the huge easy majority class (B_candidate). The minority class that actually matters has F1 = 0.18.

30. 'Near a facility' does not mean 'industrial fire' — refineries have forests and roads nearby too. Using it as a label would be circular. It is a context feature only.

31. Transparency and control: every point is explainable to a judge/operator, and there is no labelled 'risk' target to learn from.

32. Safety for a live internal demo — the agent can help you look but a human makes every consequential change. Write-actions are a deliberate later step behind confirmation.

33. Offline-first: the demo must run with no network and no key. Claude adds cost/latency/network and must degrade cleanly; it can do nothing the parser can't.

34. So it is framework-agnostic and reusable — a future FastAPI + React frontend could sit on the same functions without touching logic.

35. No dataset of confirmed industrial fires exists (India or global), and a FIRMS hotspot has no cause label. We detect anomalous departures from known patterns — screening, not confirmation.

36. 'That number is inflated by the easy majority class. Our honest metric is Class A F1 = 0.18 on a spatial holdout, and we report the leakage gap openly — the minority industrial class is hard because only ~1,900 labelled positives exist.'

37. 'We flag anomalous thermal departures near industrial infrastructure for human verification. We do not and cannot confirm fires — no ground-truth dataset exists.'

38. 'No — it's a screening layer. Recall is limited by label scarcity, not model choice. It still surfaces the right anomalies for review, and a historical FIRMS archive or GIHS would raise recall. The risk engine and investigation add operational value on top.'

39. 'No. Proximity is one of seven features and a risk signal, never a label. Forests, farms and towns are near industry too. Labels come from the VIIRS Nightfire flare catalogue.'

40. 'FIRMS confidence is a detection-quality flag, not a cause. We do use it (+8 in the risk score when high), but it can't separate industrial from natural.'

41. 'It can't — it's strictly read-only. It calls a fixed registry of query-only tools; there is no state-changing tool registered, no SQL, no DB write. Worst case is a wrong list.'

42. 'A wildfire dashboard shows where fires are. We add industrial context, a persistent-vs-natural classifier with an anomaly path, a transparent risk score, an explainable per-alert investigation, and a natural-language interface — all framed honestly as anomaly screening.'

43. 'Near-real-time, in batches. The pipeline scores FIRMS NRT and writes parquet; the architecture would extend to streaming, but that plus auth and an archive are future work.'

44. 'It's a deliberate margin: requiring >=55% for a confident call widens the review band so borderline events (~0.50-0.55) get human eyes. It's a fixed operational rule, tunable, not learned.'

45. 'They are an independent evaluation / demo set (30 curated real incidents), NOT a training class. 21/30 land as anomalies, which is the intended behaviour — real incidents match neither learned pattern.'

46. On the next load, the guard at the top of dashboard/app.py sees data/alerts.db missing and calls src.alerting.pipeline.run(fresh=True), which re-scores stage6_india_scores.parquet through risk_engine and re-inserts alerts via alert_store. The demo self-heals.

47. frp_mw=NaN flows into X; the Pipeline's SimpleImputer(strategy='median') replaces it with the training-set median before the RandomForest predicts -> predicted_label + probs + anomaly_flag. The risk engine reads frp_mw via row.get('frp_mw',0) or 0, so a missing FRP simply contributes 0 FRP points; other signals still score.

48. (1) deterministic parser didn't extract state='Odisha' (entity extraction / geo.REGIONS); (2) the tool ran but ui_action.filters wasn't built or wasn't written through state.py; (3) the panel didn't call st.rerun() / the Map page reads a different filter key than the one written.

49. Change src/features/engineer.py (compute the feature), src/model/assemble.py + train.py (add to TRAIN_FEATURES), optionally src/scoring/score_incidents.py. Re-run Stages 4->5->6 to regenerate features_stage4, stage5_*, the model and stage6_india_scores.parquet. Risk engine only changes if you want it to use the new field.

50. The architecture rule is that dashboard/ imports only src/intelligence/, so both UI and agent share one logic layer and a future non-Streamlit frontend stays possible. Fix: add/whichever query is needed to src/intelligence/queries.py (or actions.py) and call that from the page.

Night-Before-Hackathon Cheat Sheet

Say this if you only remember one thing

> **THE LINE**
> "NASA FIRMS sees heat but not cause. We add industrial + geographic context to every detection over India, classify persistent industrial sources vs natural fires with a Random Forest (India held out of training), flag the uncertain ones as anomalies for human review, score the risk with a transparent rule engine, and show prioritised, explained alerts on a GIS dashboard. It runs offline. We never claim 'confirmed fire' — no such dataset exists."

The 7 features (memorise)

`bt_kelvin | frp_mw | persistence_count | dist_nearest_facility_km | agri_season_flag | day_night_bin | acq_month`

The numbers that matter

train / val / India	270,238 / 64,864 / 705
Class A rows (labels)	`1,901 total (~0.57%) — this is why recall is hard`
Anomaly threshold	`max(prob) < 0.55`
Honest model metric	`Class A F1 = 0.18 (spatial holdout); accuracy 0.98 is inflated`
India anomaly rate	`59 / 705 = 8.4%`
Severity cutoffs	`65 / 40 / 20`
Facilities	`72,624 (WRI + OSM)`
Incidents	`30 scored, 21 anomaly-flagged — evaluation set, not training`

The 6 things NOT to say

- Do NOT say "we detect industrial fires" — say "we flag anomalous thermal departures for review".
- Do NOT say "98% accurate" without adding "Class A F1 is 0.18, the honest number".
- Do NOT say the agent "does things" — it is read-only.
- Do NOT say we trained on Indian data — India is held out.
- Do NOT say "near a facility means industrial" — proximity is a feature, not a label.
- Do NOT present a `[PLANNED]` feature (Investigation page, Facilities page, agent) as finished — say "designed, in progress".

If a demo breaks

- Map/agent won't update -> the filter didn't reach `session_state`; refresh the page.
- No alerts -> delete `data/alerts.db` and reload; it re-seeds automatically.
- Agent gives a weird answer offline -> the deterministic parser missed an entity; rephrase with explicit words ("critical", state name, "last 7 days").
- Claude errors -> expected; it falls back to offline mode, keep going.

Three explanations, three lengths

- **30s:** problem (heat, not cause) + solution (context + RF + anomaly + risk + GIS).

- **1 min:** add the architecture (offline pipeline -> parquet -> rule engine -> SQLite alerts -> Streamlit) + differentiation (honest framing, no leakage, optional read-only agent).
- **3 min:** full story — data + proxy labels + 7 features + RF + 3-way eval + 0.55 anomaly + risk engine + lifecycle + dashboard + agent.

Final gut-check questions

```
Q Is this a confirmed fire?          -> No. Anomaly screening for human review.
Q Why Random Forest?                 -> Tabular, tiny positive class, explainable.
Q What are your labels?              -> Proxy from VIIRS Nightfire flare sites (5 km).
Q Model uncertain?                   -> max(prob) < 0.55 -> anomaly_flag.
Q Classification vs risk?            -> what it is  vs  how concerning (rules).
Q Nearest facility?                  -> haversine BallTree over 72,624 rows.
Q Agent needs internet?              -> No, deterministic offline baseline.
Q Agent can change alerts?           -> No, strictly read-only.
Q Trained on India?                  -> No, India is the held-out test region.
```

> **REMEMBER**
>
> Eyes -> Brain -> Threat board -> Map -> Notebook -> Control room -> Assistant. Satellite -> ML -> Risk -> GIS -> Alert store -> Dashboard -> Agent.